EAST WEST UNIVERSITY

Department of Computer Science and Engineering

PROJECT REPORT

Blood Cell Image Classification Using Convolutional Neural Networks and CBAM Attention

Group No: 10

Task 2 | Track 3: CNN + Attention

Submitted by

Student ID	Student Name
2023-1-60-023	Md.F.K.Murad
2022-2-60-135	Jalal Uddin Mohammad
2023-3-60-490	Md Julfiker Hossain

Course code: CSE 475

Submitted to: Dr. Raihan Ul Islam

Submission Date: 12 November 2026

Abstract

This project investigates automated classification of blood cell images using convolutional neural networks (CNNs), with a focus on the effect of attention mechanisms. The BloodMNIST dataset was used with 64 × 64 RGB images and eight cell categories. Four conventional baseline models—SimpleCNN, ResNet50, DenseNet121 and MobileNetV2—were evaluated in the baseline notebook. A compact CNN was then extended with Convolutional Block Attention Module (CBAM), which applies channel attention followed by spatial attention. The proposed model was compared with the same CNN without attention under a controlled training setup and three random seeds. The proposed CBAM model achieved a mean test macro-F1 of 0.9777 ± 0.0049, compared with 0.9689 ± 0.0135 for the no-attention CNN. The paired macro-F1 difference was positive on two of the three seeds, with a mean difference of +0.0087 and a standard deviation of 0.0159. Therefore, the results suggest a modest average improvement, but do not establish a consistent or statistically significant gain. The report presents the experimental setup, results and discussion without overstating the effect of attention.

Keywords: BloodMNIST, blood cell classification, CNN, CBAM, channel attention, spatial attention, macro-F1.

1. Introduction

Blood cell image classification is an important image-analysis task because different blood cell types have distinct morphological and visual characteristics. Manual examination requires specialist time and can be affected by workload and observer variability. Deep learning, particularly convolutional neural networks, can learn discriminative visual features directly from images and can support automated classification workflows.

Although a CNN can learn useful features, not all feature channels or spatial locations contribute equally to a classification decision. Attention mechanisms address this issue by assigning different weights to feature channels and image locations. This project follows Track 3 (CNN + Attention) and studies CBAM [3], a lightweight attention module that sequentially applies channel and spatial attention.

2. Problem Statement

The task is to classify a 64 × 64 RGB blood-cell image into one of eight categories. The dataset is imbalanced, and some cell categories have visually similar characteristics. A compact CNN may not emphasize the most useful feature channels and spatial regions consistently. The project therefore evaluates whether adding CBAM to the same CNN backbone improves classification performance, while also comparing the proposed model with commonly used CNN architectures. The majority class (neutrophil, 3,329 images across the full dataset) is about 2.74 times larger than the minority class (lymphocyte, 1,214 images).

3. Aim and Objectives

Aim: To develop and evaluate a CNN-based blood-cell image classifier enhanced with CBAM attention.

1. Prepare the BloodMNIST dataset using leakage-aware splitting, training-only normalization and training-only augmentation.
2. Train and compare SimpleCNN, ResNet50, DenseNet121 and MobileNetV2 as baseline models.
3. Integrate CBAM into the SimpleCNN backbone and evaluate its effect through a controlled comparison across three random seeds.

4. Dataset Description

The experiments use BloodMNIST from the MedMNIST v2 collection [2], whose images originate from the blood-cell dataset of Acevedo et al. [1]. The notebook loads the official training, validation and test partitions at image size 64. The complete dataset contains 17,092 images, each represented as a 64 × 64 RGB image, and the classification task has eight classes.

Split / property	Count / value
Training images before duplicate control	11,959
Training images after duplicate control	11,951
Validation images	1,712
Test images	3,421
Total images after duplicate control	17,084
Image dimensions	64 × 64 × 3
Number of classes	8

The eight classes are:

- Basophil
- Eosinophil
- Erythroblast
- Immature granulocytes
- Lymphocyte
- Monocyte
- Neutrophil
- Platelet

5. Data Preparation and Preprocessing

5.1 Duplicate and leakage audit

An MD5 fingerprint was calculated from the raw pixel bytes of every image. The audit identified 3 train–validation overlap groups, 5 train–test overlap groups, 1 validation–test overlap group and 1 conflicting-label group. Eight training images were removed. Validation and test samples were retained unchanged. The resulting split sizes were 11,951 training, 1,712 validation and 3,421 test images.

The notebook notes that the dataset does not provide subject identifiers; consequently, subject-wise splitting could not be performed. The duplicate audit is an image-level safeguard and should not be interpreted as subject-level leakage control.

5.2 Normalization and augmentation

Pixel values were converted from uint8 to floating point and scaled to [0, 1]. The training-set channel means were [0.7961, 0.6596, 0.6964] and standard deviations were [0.2233, 0.2546, 0.0947]. These training-only statistics were used to normalize the images. Augmentation was applied only to training samples and included horizontal and vertical flips, random rotations by multiples of 90 degrees, and mild brightness and contrast changes.

5.3 Class imbalance

Class weights were calculated from the training partition and supplied to weighted cross-entropy loss. This gives relatively greater loss weight to less frequent classes. The class weights in class order were approximately [1.762, 0.686, 1.377, 0.737, 1.760, 1.504, 0.641, 0.909].

6. Baseline Models

Four baseline architectures were trained from scratch using the same cleaned data split and evaluation protocol. SimpleCNN served as the compact reference backbone. ResNet50, DenseNet121 and MobileNetV2 provided comparisons against deeper or more parameter-efficient established architectures. The ResNet50 stem was adapted for 64 × 64 inputs by using a 3 × 3 convolution with stride 1 and removing the initial max-pooling operation. The architectures follow He et al. [5] (ResNet50), Huang et al. [6] (DenseNet121) and Sandler et al. [7] (MobileNetV2).

Model	Parameters (M)	Test accuracy	Test macro-F1	Test weighted-F1
SimpleCNN	0.391	0.9600	0.9535	0.9599
ResNet50	23.517	0.9728	0.9714	0.9728
DenseNet121	6.962	0.9702	0.9672	0.9703
MobileNetV2	2.234	0.9734	0.9722	0.9732

ResNet50 had the highest validation macro-F1 (0.9758) and was therefore selected as the best baseline according to the validation-based selection rule. MobileNetV2 had the highest test macro-F1 among the four baseline models (0.9722); test results are reported for evaluation and were not used to select the best baseline.

All four baselines were trained once, with seed 42. Differences of about 0.005 in macro-F1 (roughly 17 of the 3,421 test images) are therefore within the range that a different seed could change, and the ranking of ResNet50, MobileNetV2 and DenseNet121 should not be over-interpreted. In addition, the ResNet50 stem was adapted for 64 × 64 inputs whereas DenseNet121 and MobileNetV2 use their standard stems, so training times are not directly comparable across architectures.

Full test metrics of the baselines (macro and weighted averages; P = precision, R = recall):

Model	Acc.	Macro-P	Macro-R	Macro-F1	Wtd-P	Wtd-R	Wtd-F1	ROC-AUC
ResNet50	0.9728	0.9750	0.9687	0.9714	0.9734	0.9728	0.9728	0.9989
MobileNetV2	0.9734	0.9698	0.9753	0.9722	0.9738	0.9734	0.9732	0.9990
DenseNet121	0.9702	0.9681	0.9669	0.9672	0.9708	0.9702	0.9703	0.9985
SimpleCNN	0.9600	0.9579	0.9514	0.9535	0.9613	0.9600	0.9599	0.9986

Training cost of the baselines:

Model	Training time (min)	Model size (MB)	Epochs run	Best epoch
ResNet50	13.84	94.38	14	7
MobileNetV2	2.22	9.17	30	27
DenseNet121	2.25	28.42	15	8
SimpleCNN	0.49	1.58	12	5

SimpleCNN — test evaluation

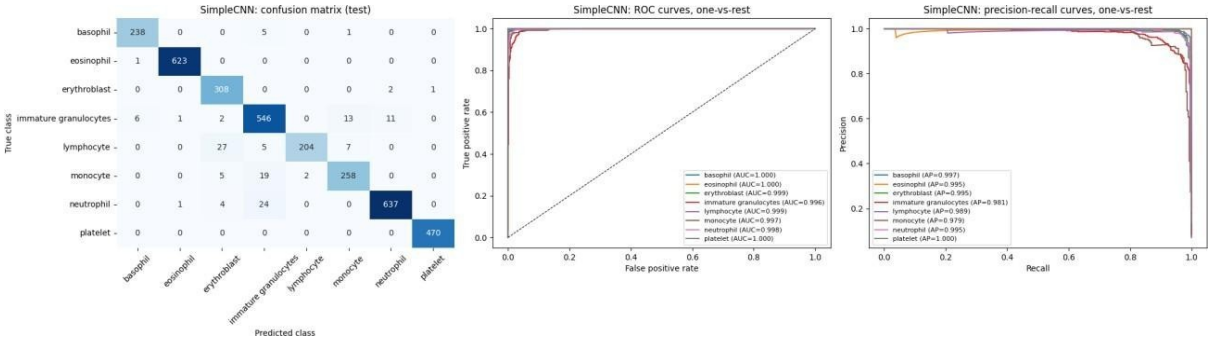


Figure 1. SimpleCNN test confusion matrix, one-vs-rest ROC curves and precision–recall curves.

ResNet50 — test evaluation

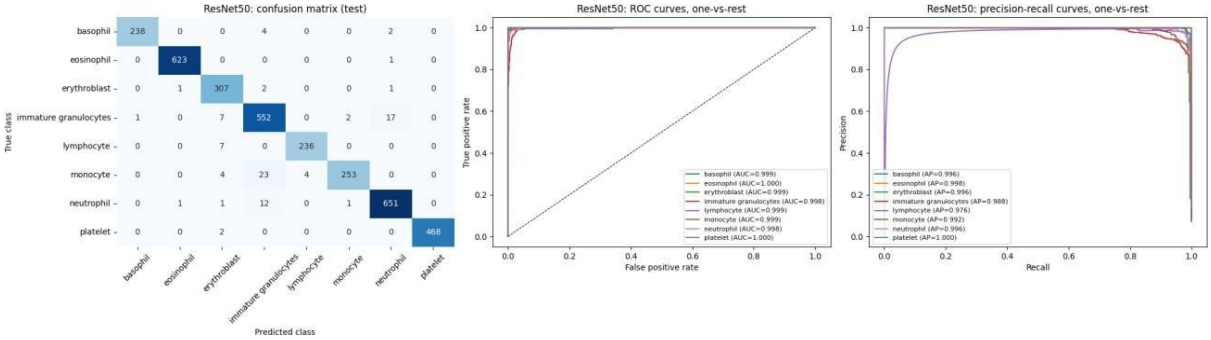


Figure 2. ResNet50 test confusion matrix, one-vs-rest ROC curves and precision–recall curves.

DenseNet121 — test evaluation

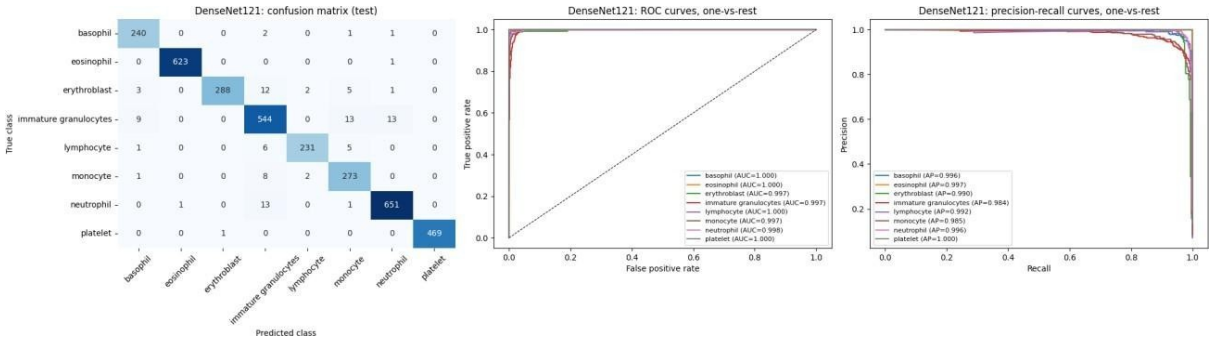


Figure 3. DenseNet121 test confusion matrix, one-vs-rest ROC curves and precision–recall curves.

MobileNetV2 — test evaluation

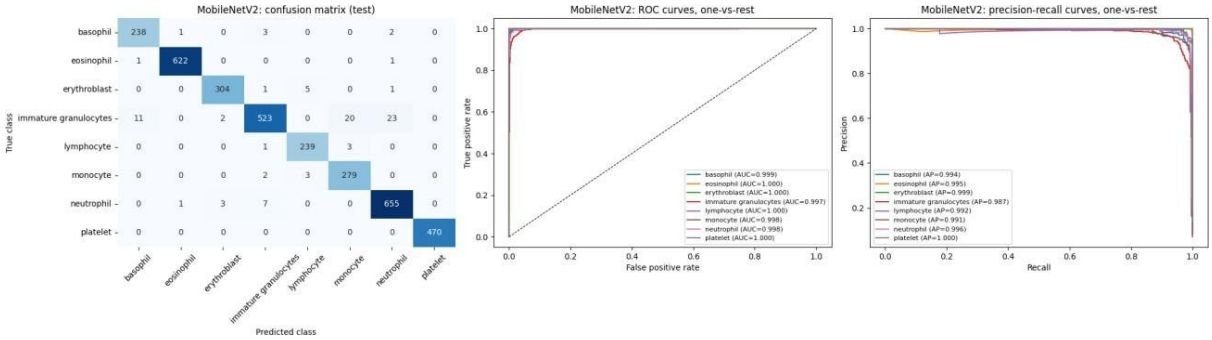


Figure 4. MobileNetV2 test confusion matrix, one-vs-rest ROC curves and precision–recall curves.

The confusion matrices show correct predictions and class-specific errors. The ROC and precision–recall curves summarize one-vs-rest discrimination for each class. These figures should be interpreted together with macro-averaged metrics because strong overall performance can conceal weaker performance for individual classes.

7. Proposed CNN with CBAM Attention

7.1 CNN backbone

The backbone contains four convolutional blocks with channel widths 32, 64, 128 and 256. Each block consists of a 3 × 3 convolution, batch normalization, ReLU activation and 2 × 2 max-pooling. The feature maps are passed through global average pooling, dropout with probability 0.3, and a fully connected layer that produces eight class scores.

CBAM was selected because it combines channel and spatial attention in a lightweight module. Unlike channel-only attention mechanisms such as Squeeze-and-Excitation (SE) [4], CBAM can refine both the importance of feature channels and the locations of relevant visual patterns. This is useful for blood-cell images, where classification may depend on subtle differences in cell shape, texture and internal structures. CBAM also introduces only a small number of additional parameters to our CNN, making it suitable for the compact architecture used in this project.

7.2 CBAM module

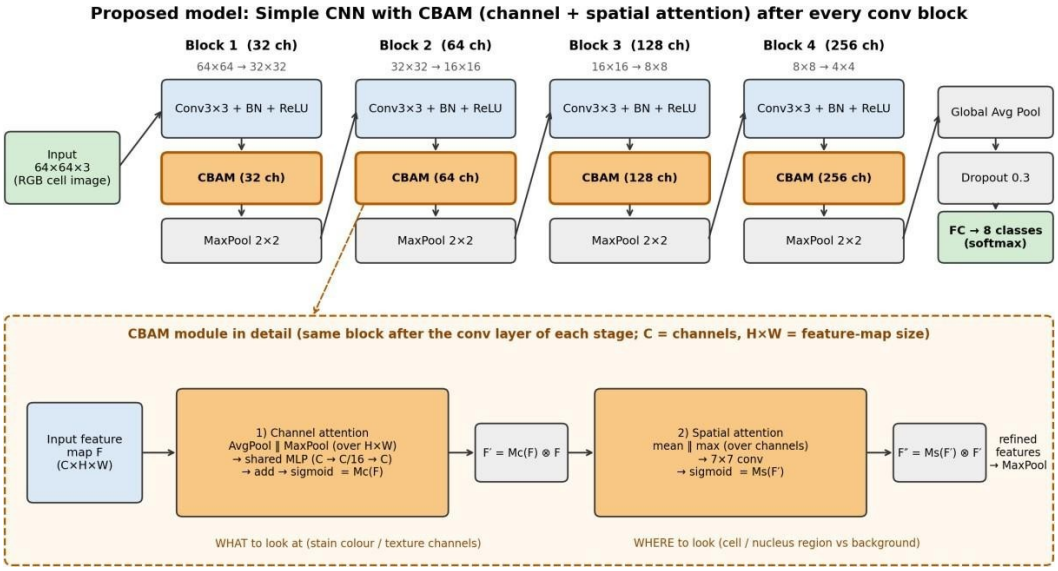


Figure 5. Proposed SimpleCNN architecture with CBAM channel and spatial attention modules.

CBAM [3] is a hybrid attention mechanism consisting of two sequential components. Channel attention estimates which feature channels are important. It uses adaptive average pooling and adaptive max pooling, passes both descriptors through a shared small multilayer mapping, combines the outputs and applies a sigmoid gate.

Spatial attention then computes channel-wise average and maximum maps, concatenates them, applies a 7 × 7 convolution and sigmoid, and produces a spatial attention map.

In the proposed network, CBAM is inserted after the ReLU activation in each of the four convolutional blocks. Channel attention can emphasize informative feature patterns, while spatial attention can emphasize relevant image locations and reduce the contribution of less useful background regions.

7.3 Model size

Model	Parameter count	Approx. checkpoint size
SimpleCNN (no attention)	391,432	1.579 MB
Proposed CNN + CBAM	402,832	1.629 MB
SimpleCNN + SE (additional attention comparison)	402,952	1.630 MB

CBAM adds approximately 11,400 parameters to the 391,432-parameter SimpleCNN backbone. This is a relatively small increase in model size. The optional SE model applies channel attention only and is included as an additional attention comparison in the experiment.

7.4 Design choices

Each part of the network has a specific role. The convolution layers learn local patterns such as edges, granules and nucleus shape; batch normalization stabilizes training; ReLU adds non-linearity; max-pooling halves the feature-map size so that deeper layers see larger image regions at low computational cost; and global average pooling replaces a large fully connected layer, which keeps the parameter count low. Dropout (0.3) reduces over-fitting, and class-weighted cross-entropy

prevents the model from favouring frequent classes. Four blocks with 32 to 256 channels keep the model small enough to train from scratch on about 12,000 images. CBAM is inserted in every block so that both low-level features (colour, edges) and high-level features (shape, texture) are refined, and a reduction ratio of 16 keeps the attention layers small.

7.5 Why we expected CBAM to help

In a plain CNN every feature channel and every pixel position has the same influence, although the cell occupies only part of the 64 × 64 crop and different classes are distinguished by different cues, such as nucleus shape, granule texture and stain colour. CBAM learns a weight for each channel and each position, so the cues that matter for a given cell can be emphasized and the plain background reduced, for about 11,400 additional parameters (about 2.9% of the backbone). We therefore expected the CNN with CBAM to outperform the same CNN without attention, especially on visually similar classes. We did not expect it to outperform the larger baselines by design, since, for example, ResNet50 has about 58 times more parameters; the comparison in Section 10 tests this expectation.

8. Experimental Setup

Setting	Configuration
Input	64 × 64 RGB
Initialization	All models trained from scratch
Optimizer	AdamW
Learning rate	0.001
Weight decay	0.0001
Loss	Class-weighted cross-entropy
Batch size	128
Maximum epochs	30
Early stopping	Patience of 7, based on validation macro-F1
Learning-rate schedule	Constant learning rate (cosine schedule disabled)
Random seeds	42, 7 and 123 (attention comparison); 42 only (baselines)
Hardware	CUDA-enabled GPU (notebook runtime)
Model selection	Best checkpoint selected by validation macro-F1
Test usage	Final evaluation only

For the controlled attention experiment, the no-attention SimpleCNN and the CBAM model used the same backbone, data partitions, preprocessing, augmentation policy, loss, optimizer settings and seed list. The training data order was controlled for corresponding models under the same seed. The optional SE model was also trained for the three seeds.

9. Evaluation Metrics

Accuracy measures the fraction of correctly classified test images. Precision measures the fraction of predicted members of a class that are correct, while recall measures the fraction of actual members of a class that are identified correctly. The F1-score is the harmonic mean of precision and recall. Macro averages assign equal weight to each class, whereas weighted averages account for class support. Macro-F1 is emphasized because the dataset is imbalanced. The notebook also reports one-vs-rest

macro ROC-AUC, macro PR-AUC, Matthews correlation coefficient (MCC), per-class precision/recall/F1, confusion matrices and ROC/precision–recall curves.

10. Experimental Results

10.1 Controlled comparison: CNN without attention vs CNN with CBAM

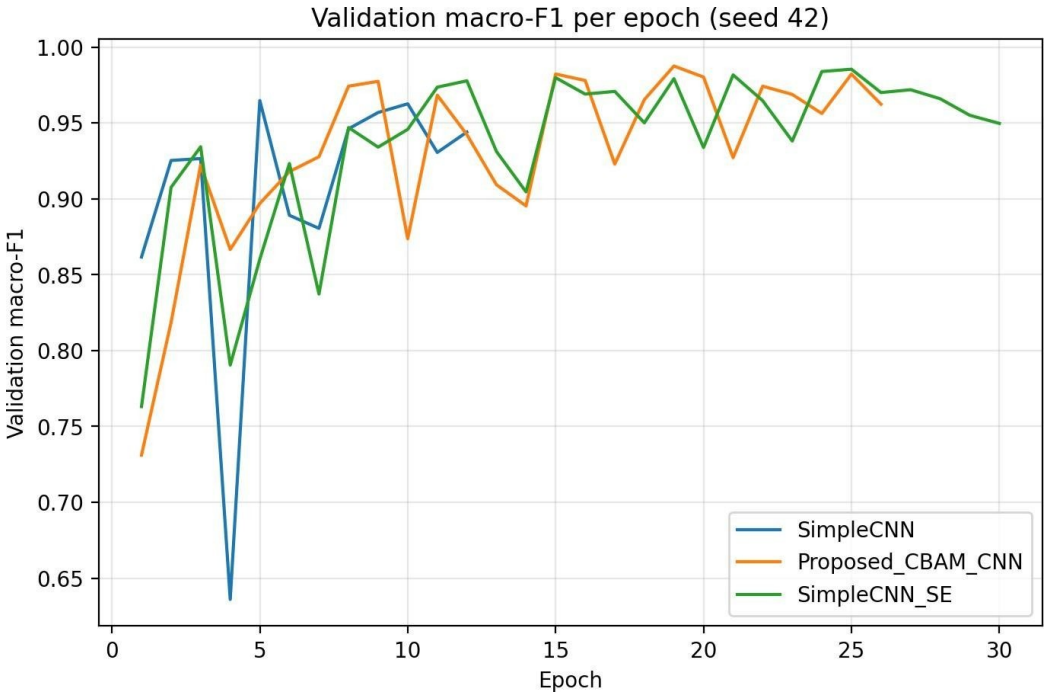


Figure 6. Validation macro-F1 across epochs for the three attention-comparison models (seed 42).

Metric (mean ± standard deviation over 3 seeds)	SimpleCNN	Proposed CNN + CBAM	SE attention
Test accuracy	0.9718 ± 0.0104	0.9777 ± 0.0045	0.9770 ± 0.0070
Test macro-F1	0.9689 ± 0.0135	0.9777 ± 0.0049	0.9771 ± 0.0072
Test weighted-F1	0.9718 ± 0.0104	0.9777 ± 0.0045	0.9771 ± 0.0070
Macro ROC-AUC (OvR)	0.9989 ± 0.0003	0.9990 ± 0.0002	0.9990 ± 0.0002
Macro PR-AUC	0.9937 ± 0.0019	0.9941 ± 0.0014	0.9941 ± 0.0020
MCC	0.9672 ± 0.0121	0.9740 ± 0.0053	0.9732 ± 0.0081

The mean test macro-F1 of the CBAM model was 0.9777, compared with 0.9689 for the no-attention CNN. The mean paired difference (CBAM minus SimpleCNN) was +0.0087, with a standard deviation of 0.0159. CBAM achieved a higher macro-F1 in two of the three seeds, while the difference was negative for seed 7. Thus, the average result favours CBAM, but the improvement was not consistent across all seeds and is not sufficient to claim a definitive improvement.

Seed	SimpleCNN macro-F1	CBAM macro-F1	CBAM − SimpleCNN
42	0.9535	0.9803	+0.0268
7	0.9749	0.9720	−0.0029
123	0.9784	0.9807	+0.0022

10.2 Comparison with baseline architectures

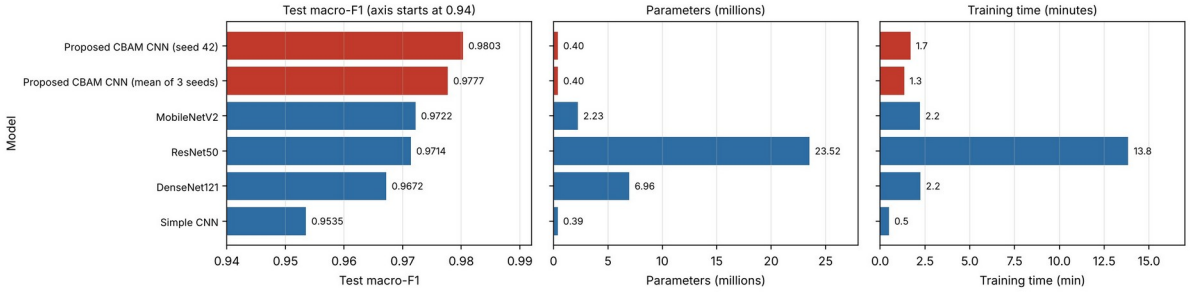


Figure 7. Test macro-F1, training time and parameter-count comparison across the proposed and baseline models.

Model	Validation macro-F1	Test accuracy	Test macro-F1	Parameters (M)
SimpleCNN	0.9647	0.9600	0.9535	0.391
ResNet50	0.9758	0.9728	0.9714	23.517
DenseNet121	0.9672	0.9702	0.9672	6.962
MobileNetV2	0.9721	0.9734	0.9722	2.234
Proposed CBAM CNN (seed 42)	0.9875	0.9795	0.9803	0.403
Proposed CBAM CNN (3-seed mean)	0.9826	0.9777	0.9777	0.403

Among the baseline models, ResNet50 was selected by validation macro-F1. The three-seed mean of the proposed CBAM CNN achieved a test macro-F1 of 0.9777, while the best baseline by validation selection (ResNet50) achieved 0.9714 on the test set. The CBAM model is substantially smaller in parameter count than ResNet50. However, the fair test of the attention mechanism is the paired comparison against the same SimpleCNN backbone, not the comparison against different architectures. The baselines were trained once (seed 42) whereas the CBAM figure is a three-seed mean, and the ResNet50 stem was adapted for 64 × 64 inputs while the other baselines use their standard stems, so the training-time and parameter comparison in Figure 7 is indicative only.

10.3 Per-class observations

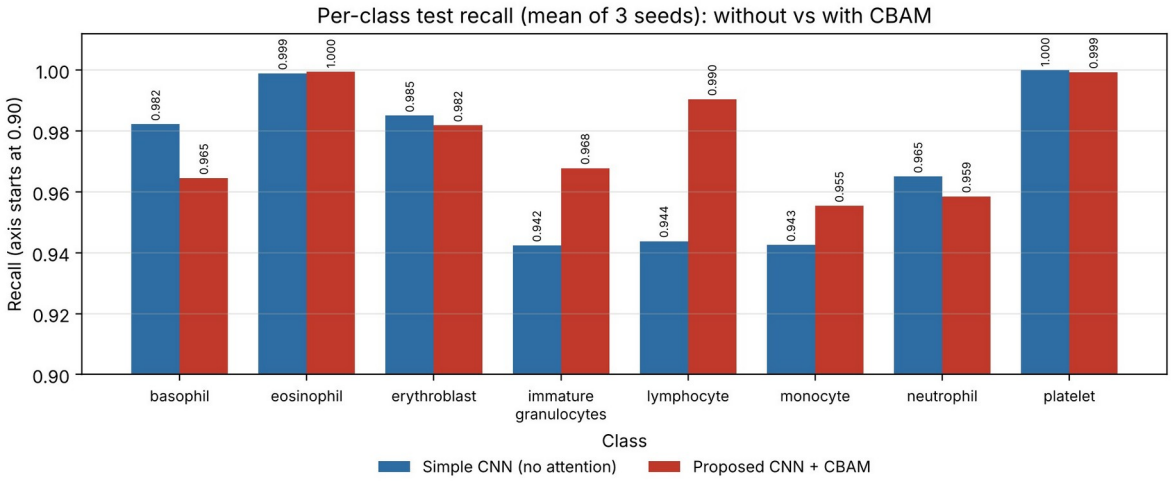


Figure 8. Per-class test recall comparison averaged over the three seeds.

Class	SimpleCNN recall	CBAM recall	Difference
Basophil	0.9822	0.9645	−0.0178
Eosinophil	0.9989	0.9995	+0.0005
Erythroblast	0.9850	0.9818	−0.0032
Immature granulocytes	0.9424	0.9678	+0.0253
Lymphocyte	0.9438	0.9904	+0.0466
Monocyte	0.9425	0.9554	+0.0129
Neutrophil	0.9650	0.9585	−0.0065
Platelet	1.0000	0.9993	−0.0007

Mean per-class recall across seeds indicates that CBAM improved recall for immature granulocytes, lymphocytes and monocytes, while recall decreased for basophils, erythroblasts, neutrophils and platelets. These mixed class-level effects reinforce the need to avoid describing attention as uniformly beneficial. The gains for lymphocytes and immature granulocytes are largely driven by seed 42, in which the SimpleCNN run was unstable (best epoch 5, early stop at epoch 12, and a lymphocyte recall of 0.8395 in the baseline notebook).

11. Discussion

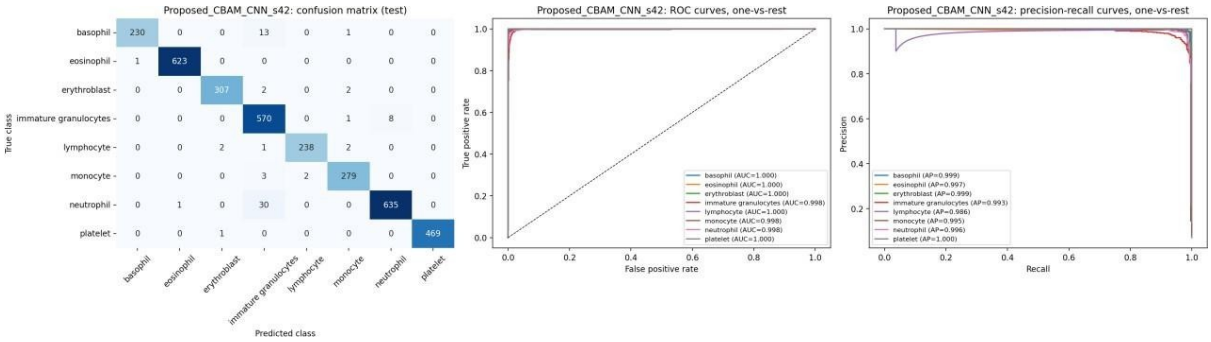


Figure 9. Confusion matrix, one-vs-rest ROC curves and precision–recall curves for the proposed CBAM CNN (seed 42).

The experiments show that the compact CBAM model is competitive with the tested baseline architectures. In the controlled three-seed experiment, CBAM produced a higher average macro-F1 than the same CNN without attention. However, the seed-wise differences varied: the gain was large for seed 42, slightly negative for seed 7 and small for seed 123. The observed mean gain should therefore be interpreted as a promising but preliminary result.

The per-class results suggest that attention may be especially helpful for some visually challenging categories, including lymphocytes and immature granulocytes, but it does not improve every category. The additional parameters introduced by CBAM are modest, although the CBAM model required more training time than the no-attention CNN in the reported three-seed summary (1.64 versus 0.62 minutes for seed 42; the CBAM three-seed mean was 1.30 minutes). In the seed-42 CBAM confusion matrix (Figure 9), most remaining errors involve immature granulocytes: 30 neutrophils and 13 basophils were predicted as immature granulocytes, and 8 immature granulocytes were predicted as neutrophils.

ResNet50 remains the baseline selected using validation macro-F1. MobileNetV2 has a slightly higher baseline test macro-F1 than ResNet50, but this does not change the validation-based selection rule.

The proposed CBAM model's three-seed mean test macro-F1 is higher than the selected ResNet50 baseline result, but cross-architecture comparisons should be interpreted with care because their capacity and training behaviour differ.

12. Conclusion

This project implemented a compact CNN for eight-class BloodMNIST classification and extended it with CBAM channel and spatial attention. Four baseline architectures were evaluated, and the proposed model was compared with the same CNN without attention across three random seeds. The CBAM model achieved a mean test macro-F1 of 0.9777 ± 0.0049, compared with 0.9689 ± 0.0135 for the no-attention CNN. The average paired gain was +0.0087, but CBAM was better in only two of the three seeds. The results therefore support CBAM as a promising lightweight enhancement in this experiment, not as a conclusively superior method. Further repeated experiments and statistical testing are needed before making a stronger claim.

13. References

[1] A. Acevedo, A. Merino, S. Alférez, Á. Molina, L. Boldú and J. Rodellar, “A dataset of microscopic peripheral blood cell images for development of automatic recognition systems,” Data in Brief, vol. 30, art. 105474, 2020.

[2] J. Yang, R. Shi, D. Wei, Z. Liu, L. Zhao, B. Ke, H. Pfister and B. Ni, “MedMNIST v2 – A large-scale lightweight benchmark for 2D and 3D biomedical image classification,” Scientific Data, vol. 10, art. 41, 2023.

[3] S. Woo, J. Park, J.-Y. Lee and I. S. Kweon, “CBAM: Convolutional Block Attention Module,” in Proc. European Conf. on Computer Vision (ECCV), 2018, pp. 3–19.

[4] J. Hu, L. Shen and G. Sun, “Squeeze-and-Excitation Networks,” in Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR), 2018, pp. 7132–7141.

[5] K. He, X. Zhang, S. Ren and J. Sun, “Deep residual learning for image recognition,” in Proc. IEEE CVPR, 2016, pp. 770–778.

[6] G. Huang, Z. Liu, L. van der Maaten and K. Q. Weinberger, “Densely connected convolutional networks,” in Proc. IEEE CVPR, 2017, pp. 4700–4708.

[7] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in Proc. IEEE CVPR, 2018, pp. 4510–4520.